Laboratório de Concorrência · Aula 10

João Marcos Moreira Nunes · RA 24055168-2 · Turma ESOFT-6S-N-C
Gerado em 16/10/2026, 16:56:29 · tempo registrado no cronômetro: 53:47

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação, as pessoas trabalham juntas para alcançar um objetivo comum, como colegas que organizam um evento dividindo as tarefas. Na competição, participantes disputam um recurso ou resultado limitado, como duas pessoas tentando ser as primeiras na fila de atendimento. Na computação, cooperação envolve threads que se comunicam ou dividem trabalho; competição ocorre quando disputam o acesso a um recurso compartilhado.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O valor esperado é 2.000.000, pois são duas threads incrementando 1.000.000 de vezes cada. Porém, sem sincronização, o resultado real pode ser menor devido à condição de corrida.

1ª execução (n = 1.000.000)

Não executado neste preenchimento; registrar o valor real da 1ª execução no OneCompiler.

2ª execução (n = 1.000.000)

Não executado neste preenchimento; registrar o valor real da 2ª execução no OneCompiler.

3ª execução (n = 1.000.000)

Não executado neste preenchimento; registrar o valor real da 3ª execução no OneCompiler.

Saída do programa

```
Saída esperada: esperado: 2000000; obtido: um valor que pode ser menor que 2000000 por causa da condição de corrida. O valor exato depende da execução e precisa ser copiado do OneCompiler.
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

O valor esperado é 2.000.000, mas a versão sem sincronização pode imprimir um valor menor. Os resultados podem variar entre execuções porque as threads intercalam suas leituras e gravações de maneiras diferentes; nem toda execução necessariamente exibirá a perda de atualizações.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Exemplo: o contador vale 5. A lê 5 e B também lê 5. A soma 1 e calcula 6; B soma 1 e também calcula 6. A grava 6 e B depois grava 6. O contador termina em 6, embora tenham ocorrido duas tentativas de incremento; uma atualização foi perdida porque ambas partiram do mesmo valor antigo.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, o programa pode até imprimir 2000 em algumas execuções, porque a janela para a corrida é menor, mas isso não prova que o código está correto. Em produção, a concorrência e a carga variam e o erro pode aparecer de forma intermitente, sendo difícil de reproduzir e causando dados incorretos.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

BBAAAB → 6

BBBAAA → 7

AABBAB → 6

AABBBA → 4

Saída do programa

```
Saída determinística do código:
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
```

```
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis, pois escolhemos 3 das 6 posições para os passos de A: C(6,3) = 20. Aparecem 4 resultados diferentes: 4, 6, 7 e 8. Apenas uma ordem, AAABBB, produz 8, que é o resultado correto quando A termina antes de B.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem é AABBBA. A lê 3 e calcula 4, mas ainda não grava. B lê 3, calcula 6 e grava 6. Em seguida, A grava seu valor antigo calculado, 4, sobrescrevendo o valor de B. Os passos restantes de B não mudam esse resultado nessa ordem. Assim, a atualização de B é sobrescrita e o TOTAL termina em 4.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão sem sincronização provavelmente será a mais rápida, mas pode produzir um valor incorreto. synchronized e AtomicInteger devem produzir 2.000.000; seus tempos variam, pois a sincronização tem custo.

Saída do programa

```
Resultados esperados: esperado = 2000000; sem sincronização = valor possivelmente menor que 2000000;
synchronized = 2000000; AtomicInteger = 2000000. Os tempos em milissegundos variam por execução e devem ser
copiados do OneCompiler para registrar medições reais.
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger devem chegar a 2.000.000. A versão sem sincronização pode perder incrementos. As versões corretas costumam ser mais lentas porque precisam coordenar o acesso concorrente: synchronized permite que apenas uma thread por vez execute a seção crítica, enquanto AtomicInteger usa operações atômicas com custo de coordenação.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. AtomicInteger torna atômica a atualização de cada saldo isoladamente, mas a transferência envolve duas operações que precisam ser tratadas como uma única transação: debitar uma conta e creditar a outra. Seria necessário proteger a transferência inteira com um

mecanismo de sincronização/transação adequado, garantindo consistência e evitando interleavings incorretos.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Simultaneamente, é possível que cada thread segure um recurso e espere pelo outro. Nesse caso, as mensagens “A terminou” e “B terminou” não aparecem antes de o programa detectar o impasse.

Saídas (antes e depois da correção)

```
Antes da correção: espera-se “IMPASSE: 2 threads presas”; as mensagens de conclusão não aparecem. Depois de fazer B adquirir X e depois Y, espera-se “A terminou” e “B terminou” e, ao final, “sem impasse”. A ordem das duas mensagens de conclusão pode variar.
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A adquire X e depois tenta adquirir Y. A thread B adquire Y e depois tenta adquirir X. A fica esperando Y, que está com B; B fica esperando X, que está com A. Como nenhuma libera o recurso que possui antes de obter o outro, as duas ficam bloqueadas: isso é deadlock.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Uma ordem global (por exemplo, sempre adquirir X antes de Y) impede que uma thread segure Y enquanto espera X, quebrando a condição circular necessária ao deadlock. Isso pode ocorrer em sistemas que bloqueiam duas contas, arquivos, linhas de banco de dados ou recursos compartilhados ao mesmo tempo.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 1 trabalhador, aproximadamente 600 ms; com 3, aproximadamente 200 ms; com 6, aproximadamente 100 ms. São estimativas ideais para seis tarefas de 100 ms; na prática há overhead e os tempos variam.

Tempo com 1 trabalhador

Estimativa: ~600 ms (medir no OneCompiler)

Tempo com 3 trabalhadores

Estimativa: ~200 ms (medir no OneCompiler)

Tempo com 6 trabalhadores

Estimativa: ~100 ms (medir no OneCompiler)

Saída do programa

```
Saída variável: cada trabalhador informa as tarefas que executou e, ao final, o programa imprime o número de trabalhadores e o tempo total. Tempos teóricos aproximados: 1 trabalhador ~600 ms; 3 trabalhadores ~200 ms; 6 trabalhadores ~100 ms. Registrar a saída e os tempos reais após executar.
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as seis tarefas de 100 ms são feitas uma após a outra, levando perto de 600 ms. Com 3 trabalhadores, podem ser processadas em duas rodadas, perto de 200 ms; com 6, em paralelo, perto de 100 ms. Os tempos reais incluem overhead. O programa distribui tarefas por um canal e cada tarefa é independente; não há contador compartilhado sendo atualizado por todos, e o WaitGroup apenas espera o fim dos trabalhadores.

Saídas (antes e depois da troca)

```
Antes: o canal sem buffer bloqueia no envio ch ← 1, pois o mesmo goroutine ainda não chegou à operação de recebimento; ocorre deadlock e o Go normalmente informa “all goroutines are asleep - deadlock!”. Depois de trocar para make(chan int, 1), o envio fica armazenado no buffer e o recebimento imprime 1.
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Um canal sem buffer só conclui o envio quando outro goroutine recebe a mensagem. Como a mesma goroutine tenta enviar antes de executar o recebimento, fica bloqueada e nunca chega ao <-ch. Com buffer de tamanho 1, o valor 1 pode ficar armazenado no canal; o envio termina e, em seguida, o programa recebe e imprime 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

A primeira versão deve falhar na compilação: a closure da thread tenta usar e modificar contador sem transferir sua propriedade de forma válida. Com move, a thread recebe a variável por valor e a main não é atualizada.

Saídas (o erro e o resultado com move)

```
Primeira versão: erro de compilação por tentar usar contador dentro da thread sem uma
transferência/compartilhamento permitido pelo sistema de propriedade do Rust. Com move: imprime 0, porque o
inteiro i32 é copiado para a closure e a thread altera sua própria cópia. A mensagem exata do compilador pode
variar.
```

Saída do programa

```
Saída esperada: contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a closure passa a possuir o valor capturado. Como i32 é um tipo Copy, a thread recebe uma cópia do número inicial 0. Ela incrementa essa cópia, enquanto a variável contador da função main permanece 0; por isso o programa imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a condição de corrida geralmente é descoberta durante os testes ou em produção, porque o programa compila e o problema depende do interleaving das threads. Em Rust, a tentativa de compartilhar a variável de forma inválida é barrada na compilação. A vantagem é detectar muitos erros antes de executar; o custo é precisar aprender e respeitar as regras de propriedade, empréstimo e sincronização do Rust.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem síncrona inicial deve ser A e D; depois as microtarefas de Promise são executadas antes dos callbacks de timer, então C vem antes de B. E aparece depois do bloqueio e do timer de 10 ms.

Saída do programa

```
Ordem esperada:
A
D
C
B
E: timer de 10 ms rodou depois de aproximadamente 300 ms ou mais. O tempo exato varia conforme o ambiente.
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem é A, D, C, B e depois E. A e D são impressos imediatamente no código principal. C é uma microtask de Promise e é executada antes dos callbacks de setTimeout. B, apesar de ter atraso 0, é um callback de timer e só roda depois que a pilha atual e as microtasks pendentes são processadas.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O callback de timer inicia e executa ocupado(300), que bloqueia a única thread do event loop por cerca de 300 ms. O timer de 10 ms não pode ser atendido enquanto o event loop está bloqueado; ele roda apenas quando o controle é devolvido. Em um servidor Node.js, cálculos síncronos pesados atrasam o atendimento de todos os outros pedidos e timers; é melhor dividir o trabalho ou usar workers/execução assíncrona apropriada.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O programador, com sincronização

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida: as duas solicitações leem que há uma vaga, ambas tentam comprá-la e o sistema vende duas vezes o mesmo lugar. Para evitar isso, a verificação e a reserva devem ser uma operação atômica/transacional no servidor ou banco de dados, com bloqueio/controle de concorrência e validação da disponibilidade no momento da confirmação.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais surpreende é que uma operação aparentemente simples, como incrementar uma variável, pode perder atualizações dependendo da ordem em que as threads executam.